

บทที่ 4

การเข้ารหัสข้อมูล

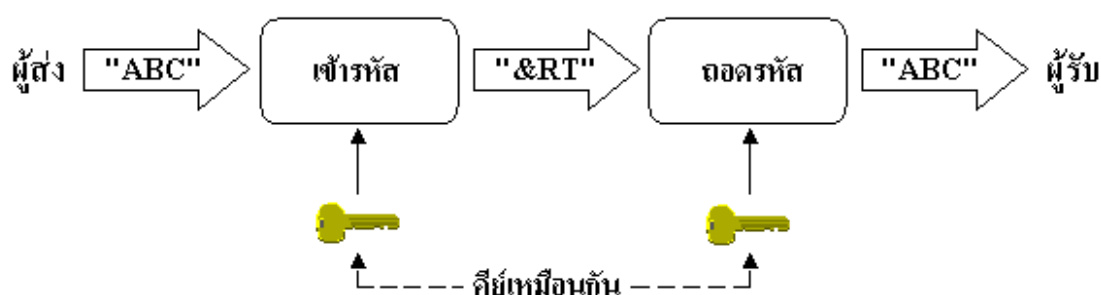
4.1 ระบบของการเข้ารหัสข้อมูล

การเข้ารหัสข้อมูลคือการทำให้ข้อมูลที่ต้องการเป็นความลับ ซึ่งถือเป็นส่วนสำคัญในระบบข้อมูลปัจจุบัน โดยอาศัยหลักการของการเข้ารหัส (Encryption) และการถอดรหัส (Decryption)

- การเข้ารหัสเป็นการเปลี่ยนรูปข้อมูลโดยผ่านรูปแบบและกระบวนการแปรรูปของข้อมูล ทำให้ข้อมูลที่ส่งมีรูปแบบที่ไม่เหมือนเดิมเพื่อให้ข้อมูลเป็นความลับ
- การถอดรหัสเป็นการแปลงข้อมูลที่ได้ผ่านการเข้ารหัสให้กลับมาเป็นข้อมูลเดิม ซึ่งการเข้ารหัสและการถอดรหัส จะถูกควบคุมโดยกุญแจหรือที่เรียกว่า “Key”

4.1.1 ระบบการเข้ารหัสแบบสมมาตร

ในระบบการเข้ารหัสแบบสมมาตร (Symmetric Cryptosystem) ซึ่งสามารถเรียกว่าการเข้ารหัสแบบคีย์เดียว (Single Key Encryption) หรือการเข้ารหัสแบบกุญแจความลับ (Secret Key Encryption) ทั้งผู้รับและผู้ส่งจะต้องมีคีย์ซึ่งเป็นความลับที่เหมือนกันในการเข้าและถอดรหัสข้อมูล หากมีคีย์ที่ต่างกันก็จะทำให้ข้อมูลที่สื่อสารกันผิดพลาด ตัวอย่างการเข้ารหัสระบบนี้ได้แก่ การเข้ารหัสแบบ DES, Triple-DES และ IDEA เป็นต้น



รูปที่ 4.1 แสดงการเข้ารหัสและถอดรหัสแบบสมมาตร

ในงานบางประเภทการเข้ารหัสแบบสมมาตรอาจจะยังไม่เพียงพอ เนื่องจากการเข้ารหัสแบบสมมาตรนั้นมีปัญหาในด้านความปลอดภัยดังนี้

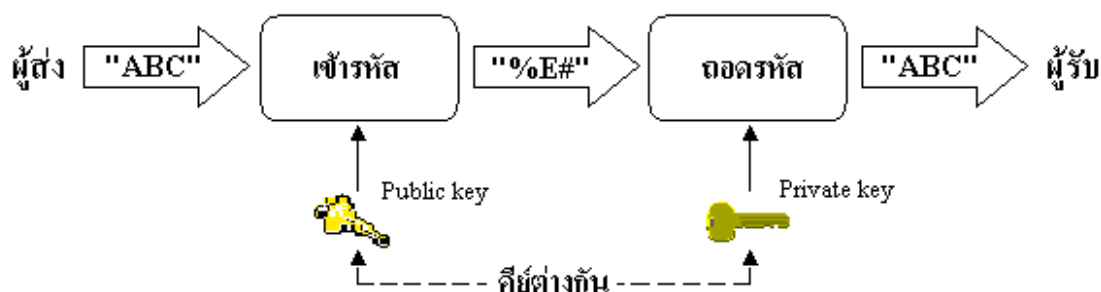
1. ระบบที่ใช้คีย์ที่เป็นความลับเพียงคีย์เดียวในการเข้ารหัสข้อมูล หากคีย์นั้นถูกขโมยไป บุคคลที่ทราบคีย์นั้นสามารถถอดรหัสข้อมูลที่ถูกรหัสไว้ได้ นอกจากนั้นอาจปลอมแปลงข้อมูลนั้นใหม่แล้วเข้ารหัสข้อมูลที่ทำการแปลงนั้นส่งไปยังผู้รับที่มีคีย์ซึ่งเป็นความลับเช่นเดียวกัน ทำให้ผู้รับนั้นได้รับข้อ

มูลที่ผิดพลาด ดังนั้นเพื่อความปลอดภัยควรเลือกคีย์ที่ยากแก่การคาดเดาและเก็บไว้อย่างปลอดภัย รวมทั้งไม่ควรใช้คีย์เดียวกันนี้ซ้ำกันหลายครั้ง

2. การเข้ารหัสแบบสมมาตรค่อนข้างอ่อนและเสี่ยงต่อการโจรกรรม เนื่องจากความยาวคีย์ที่ใช้ไม่มากพอหรืออัลกอริทึมที่ใช้นั้นค่อนข้างง่ายไม่ซับซ้อนมากนัก
3. การส่งคีย์ไปพร้อมกับข้อมูลที่เข้ารหัสนั้นอาจทำให้เกิดปัญหาได้ ซึ่งถ้าทำเช่นนั้นแล้วคีย์ที่ส่งไปด้วยนั้นจะต้องถูกส่งไปด้วยความปลอดภัยสูง วิธีที่ง่ายคือการส่งคีย์ให้กับผู้รับด้วยมือของผู้ส่งเองแต่อาจทำให้ไม่สะดวกและเสียเวลา ส่วนอีกวิธีคือการส่งคีย์ไปพร้อมกับข้อมูล แต่จะต้องทำการแบ่งคีย์ออกเป็นหลายๆ ก่อนแล้วจึงทำการส่งไปตามเส้นทางที่ต่างกัน ซึ่งแม้ว่าคีย์จะถูกดักจับได้ แต่ก็ยังเป็นเพียงคีย์ในบางส่วนไม่สามารถรู้ถึงคีย์ที่สมบูรณ์ได้

4.1.2 ระบบการเข้ารหัสแบบไม่สมมาตร

แนวคิดของการเข้ารหัสแบบไม่สมมาตร (Asymmetric Cryptosystem) หรือการเข้ารหัสแบบคีย์สาธารณะ (Public key Encryption) ได้เกิดขึ้นจากการพยายามแก้ไขปัญหของการเข้ารหัสแบบสมมาตร 2 ข้อด้วยกันคือ การจัดการคีย์และการพิสูจน์สิทธิ์ ลักษณะที่สำคัญของการเข้ารหัสคือการเข้ารหัสและเข้ารหัสโดยใช้คีย์คนละประเภทกันคือไพรเวตคีย์ (private key) ซึ่งเป็นคีย์ที่ผู้รับจะต้องเก็บเป็นความลับและพับลิคคีย์ (public key) เป็นคีย์ที่สามารถเปิดเผยให้ผู้อื่นทราบได้



รูปที่ 4.2 แสดงการเข้ารหัสและถอดรหัสแบบไม่สมมาตร

คีย์ทั้งสองนี้จะเป็นคีย์ที่ต่างกันดังรูป 4.2 ผู้ส่งจะต้องมีคีย์ของผู้รับซึ่งก็คือพับลิคคีย์ โดยเมื่อต้องการส่งผู้ส่งจะเข้ารหัสด้วยคีย์ของผู้รับ และผู้รับจะทำการถอดรหัสโดยไพรเวตคีย์ของตนเอง

จากระบบของการเข้ารหัสข้อมูลทั้งสองแบบที่ได้กล่าวมานั้น เราสามารถนำมาสรุปข้อดีและข้อเสียในระบบการเข้ารหัสข้อมูลแต่ละระบบได้ตามตารางที่ 4.1 โดยมีรายละเอียดดังนี้

การเข้ารหัสแบบสมมาตร	การเข้ารหัสแบบไม่สมมาตร
ข้อดี 1. การเข้ารหัสทำได้อย่างรวดเร็ว 2. สามารถสร้างได้ง่ายโดยฮาร์ดแวร์	ข้อดี 1. ใช้คีย์ต่างกันในการเข้ารหัสและถอดรหัส ทำให้การจัดส่งคีย์ทำได้ง่าย 2. สามารถตรวจสอบผู้ใช้ได้
ข้อเสีย 1. คีย์ของการเข้ารหัสและถอดรหัสต้องเหมือนกัน ทำให้การจัดส่งคีย์ทำได้ยาก	ข้อเสีย 1. ค่อนข้างช้าและต้องใช้เวลาคำนวณอย่างมาก

ตารางที่ 4.1 แสดงข้อดีและข้อเสียในระบบของการเข้ารหัสแต่ละระบบ

สำหรับในโครงการนี้จะใช้วิธีการเข้ารหัสข้อมูลคือ DES (Data Encryption Standard) ซึ่งเป็นระบบการเข้ารหัสแบบสมมาตร โดยทำการสร้างคีย์ DES ด้วยวิธี Diffie-Hellman ซึ่งเป็นแนวคิดในการเข้ารหัสแบบพับลิคคีย์

4.2 รูปแบบการเข้ารหัสสำหรับการสื่อสารข้อมูลในโพรโตคอลทีซีพี/ไอพี

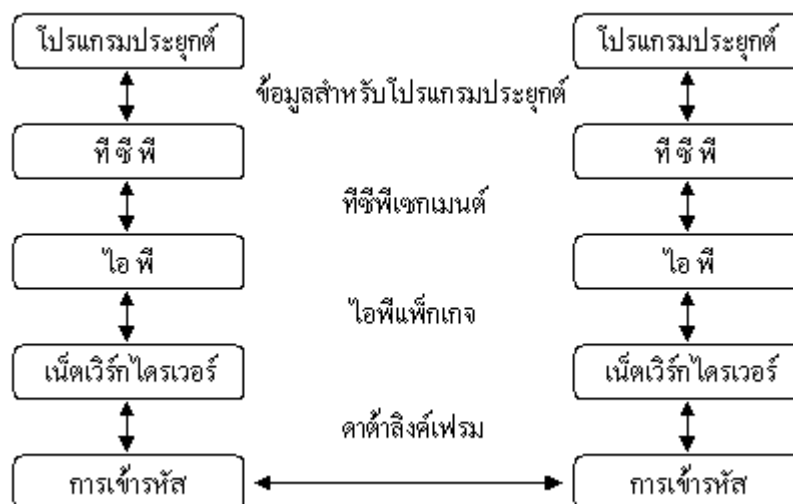
สำหรับรูปแบบการเข้ารหัสสำหรับการสื่อสารข้อมูลในโพรโตคอลทีซีพี/ไอพี แบ่งออกเป็น 4 ระดับดังต่อไปนี้

- การเข้ารหัสระดับการเชื่อมโยงข้อมูล (link level encryption)
- การเข้ารหัสระดับเครือข่าย (network level encryption)
- การเข้ารหัสระดับทรานสปอร์ต (transport level encryption)
- การเข้ารหัสระดับโปรแกรมประยุกต์ (application level encryption)

4.2.1 การเข้ารหัสระดับการเชื่อมโยงข้อมูล

การเข้ารหัสระดับการเชื่อมโยงข้อมูลเป็นการเข้ารหัสในระดับดาต้าลิงค์ โดยทั่วไปมักจะเป็นการใช้อุปกรณ์พิเศษที่เรียกว่ากล่องเข้ารหัส (encryption box) ซึ่งวิธีการนี้ถูกใช้ในระบบไซเฟอร์หรือการใช้ดีไวส์ไคร์เวอร์

วิธีการนี้จัดเป็นวิธีการที่ปลอดภัยที่สุดเนื่องจากข้อมูลถูกเข้ารหัสทั้งหมด นั่นคือทั้งข้อมูลที่ใช้งานและข้อมูลที่เป็นโครงสร้างของโพรโตคอล แต่วิธีการนี้ก็เป็นวิธีการที่มีค่าใช้จ่ายสูง เนื่องจากอุปกรณ์สำหรับการสื่อสารข้อมูลทั้งหมดจะต้องสนับสนุนการเข้ารหัสนี้ด้วย



รูปที่ 4.3 การเข้ารหัสในระดับการเชื่อมโยงข้อมูล

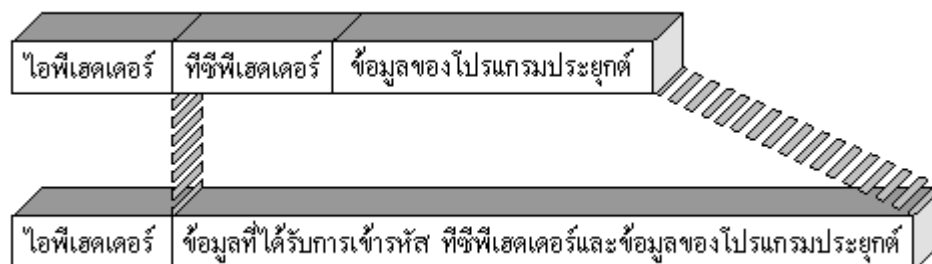
4.2.2 การเข้ารหัสระดับเครือข่าย

การเข้ารหัสข้อมูลในระดับเครือข่ายเป็นการเข้ารหัสในไอพีแพ็กเก็ต (IP Packet) โดยมีลักษณะดังรูปที่ 4.4



รูปที่ 4.4 การเข้ารหัสในระดับเครือข่าย

4.2.3 การเข้ารหัสระดับทรานสปอร์ต



รูปที่ 4.5 การเข้ารหัสในระดับทรานสปอร์ต

การเข้ารหัสข้อมูลในระดับทรานสปอร์ตเป็นการเข้ารหัสข้อมูลในส่วนของทีซีพีเซกเมนต์ (TCP Segment) ได้แก่ทีซีพีเฮดเดอร์ (TCP Header) และข้อมูลของโปรแกรมประยุกต์ ซึ่งการเข้ารหัสในลักษณะนี้จะทำให้การส่งข้อมูลไปตามเครือข่ายสามารถทำได้โดยไม่ต้องเปลี่ยนแปลงอุปกรณ์ใดๆ เนื่องจากโครงสร้างในส่วนของไอพีไม่มีการเปลี่ยนแปลง

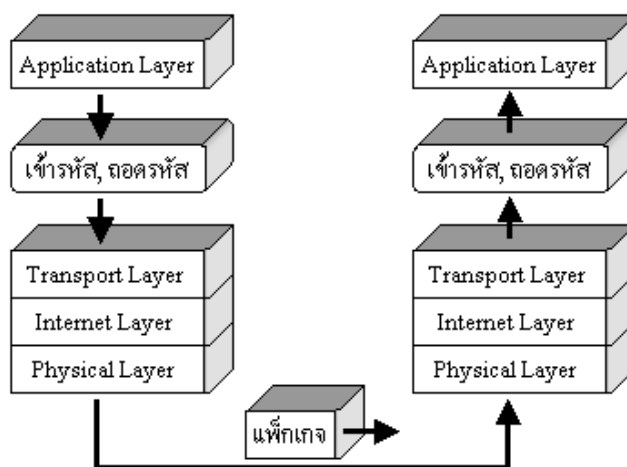
4.2.4 การเข้ารหัสระดับโปรแกรมประยุกต์

การเข้ารหัสข้อมูลในระดับโปรแกรมประยุกต์ เป็นการเข้ารหัสในลักษณะของการเข้ารหัสแบบจุดต่อจุด (end-to-end encryption) นั่นคือโปรแกรมประยุกต์จะเข้ารหัสในส่วนข้อมูลของตนเองก่อนที่จะส่งผ่านไปยังระดับล่าง ทำให้กระบวนการในระดับล่างไม่จำเป็นต้องมีการเปลี่ยนแปลงใดๆ ซึ่งวิธีการนี้ถ้าต้องการนำมาใช้กับโปรแกรมประยุกต์เดิมเช่นเทลเน็ต (telnet) หรือเอฟทีพี (FTP) จะต้องทำการแก้ไขโปรแกรมใหม่ทั้งโปรแกรมขอรับบริการและโปรแกรมสำหรับให้บริการ



รูปที่ 4.6 การเข้ารหัสในระดับโปรแกรมประยุกต์

สำหรับในโครงงานนี้ จะทำการเข้าและถอดรหัสข้อมูลในระหว่างชั้นโปรแกรมประยุกต์และชั้นทรานสปอร์ตในระบบโอเอสไอโมเดล (OSI Model) ของโพรโทคอลทีซีพี/ไอพี



รูปที่ 4.7 การเข้ารหัสบนระบบโอเอสไอโมเดลของทีซีพี/ไอพี

4.3 การเข้ารหัสแบบ DES

4.3.1 ประวัติและที่มาของ DES

ในปลายทศวรรษที่ 1960 บริษัทไอบีเอ็มได้จัดตั้งโครงการวิจัยทางการเข้ารหัสด้วยคอมพิวเตอร์ (Computer Cryptography) ซึ่งนำโดยฮอร์สท์ เฟสเทล (Horst Feistel) ซึ่งโครงการนี้เสร็จสิ้นในปี ค.ศ.1971 ซึ่งผลงานวิจัยของโครงการนี้ก็คือลูซิเฟอร์ (LUCIFER [FEIS73]) โดยมีลักษณะเป็นการเข้ารหัสข้อมูลเป็น บล็อกขนาด 64 บิตและใช้คีย์ขนาด 128 บิต ซึ่งต่อมาได้ถูกพัฒนามาขนาดของคีย์ให้ลดลงเหลือขนาด 56 บิต

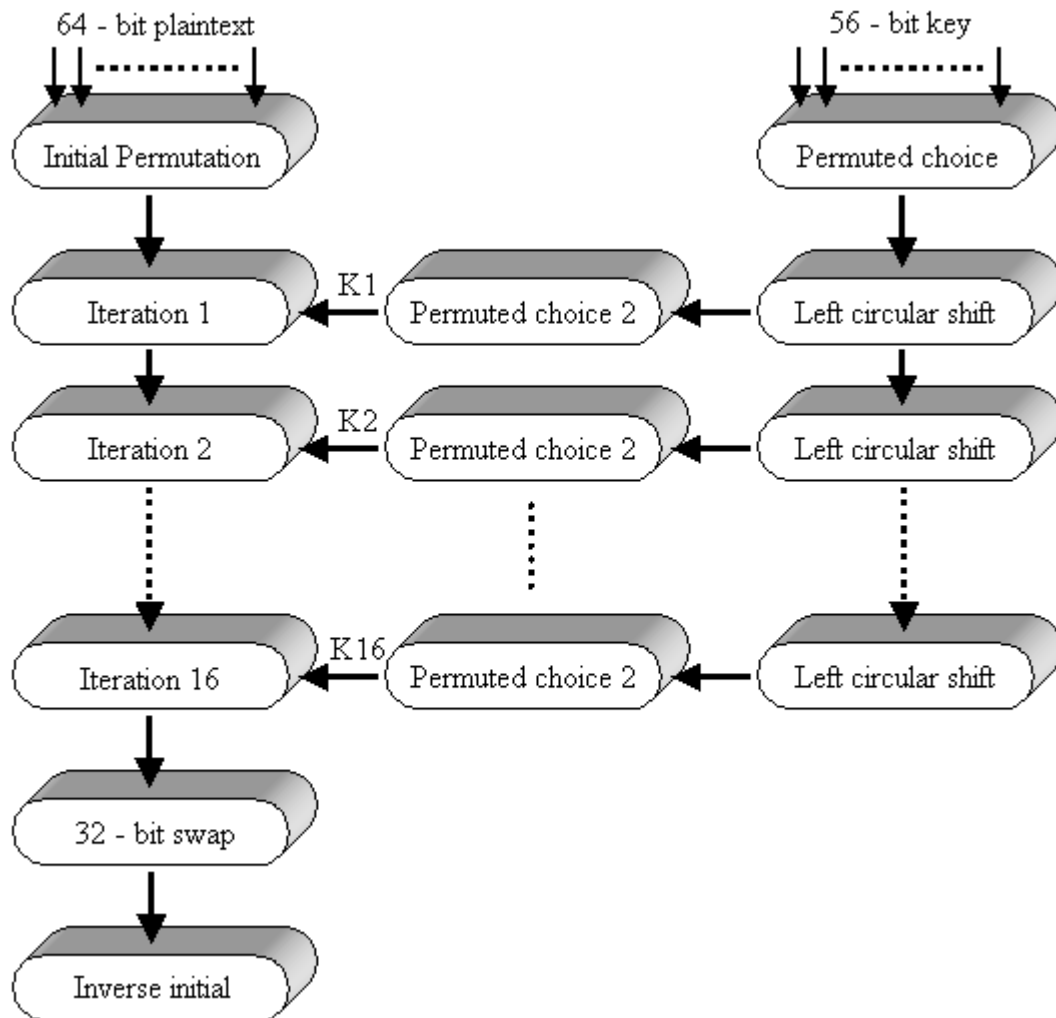
โดยอัลกอริทึมของการเข้ารหัสของลูซิเฟอร์ได้ถูกพัฒนาโดยไอบีเอ็มสำหรับ NBS (National Bureau of Standards) อัลกอริทึมนี้ได้เป็นที่รู้จักในนามของ DES (Data Encryption Standard) ถึงแม้ว่าชื่อจริงของมัน คือ DEA (Data Encryption Algorithm) ในประเทศสหรัฐอเมริกาและ DEAI (Data Encryption Algorithm-1) ในอีกหลายๆ ประเทศ

4.3.2 รายละเอียดของ DES

เป็นวิธีการเข้ารหัสที่ใช้กันอย่างแพร่หลายที่เป็นพื้นฐานบน Data Encryption Standard (DES) ที่ได้พัฒนาขึ้นในปี ค.ศ.1977 โดย National Bureau of Standards ซึ่งปัจจุบันคือ Federal Information Processing Standard 46 (FIPS PUB46) สำหรับ DES ข้อมูลจะถูกเข้ารหัสเป็นบล็อกขนาด 64 บิตซึ่งใช้คีย์ขนาด 56 บิต โดยวิธีการจัดการกับข้อมูล 64 บิตที่เข้ามาเพื่อแปลงเป็น 64 บิตข้อมูลออกไปและใช้คีย์ตัวเดียวกันนี้ในการถอดรหัส

แม้ว่า DES ถูกนำมาใช้ตั้งแต่ช่วงทศวรรษที่ 70 (ค.ศ.1960 – 1970) และได้รับการตอบรับอย่างดีจาก เหล่านักวิเคราะห์รหัส (Cryptanalysis) อย่างแพร่หลาย แต่ก็มีข้อถกเถียงกันเป็นอย่างมากถึงเรื่อง DES นั้นจะปลอดภัยได้หรือไม่และมีความปลอดภัยมากน้อยแค่ไหน แต่จนถึงในปัจจุบันเราก็ยังไม่พบจุดบกพร่องของ DES ตามเอกสารที่ตีพิมพ์เป็นสาธารณะ แม้ว่าจะใช้คีย์เพียงไม่กี่บิตก็ตาม ในทางตรงกันข้ามแนวคิดแบบ IDEA กลับใช้คีย์แบบขนาด 128 บิต (ซึ่งมีขนาดเป็น 2 เท่าของ DES) และได้รับการตอบรับจากสาธารณะตั้งแต่ทศวรรษ 90 (ค.ศ. 1980 – 1990) IDEA มีความปลอดภัยมากกว่า DES และสามารถประมวลผลได้เร็วกว่า DES อย่างไรก็ตาม IDEA ยังต้องรอการตรวจสอบจากผู้เชี่ยวชาญถึงเรื่องช่องโหว่ของความปลอดภัย

การทำงานของ DES จะมีลักษณะคือข้อมูลที่เข้ามาในส่วนฟังก์ชันของการเข้ารหัสจะมี 2 ส่วนด้วยกันคือ ข้อมูลที่ยังไม่ถูกเข้ารหัสขนาด 64 บิตและคีย์ซึ่งมีขนาด 56 บิต ซึ่งรูปในด้านซ้ายมือจะแสดงขั้นตอนจัดการกับข้อมูลที่ยังไม่เข้ารหัส โดยสามารถแบ่งย่อยๆได้อีก 3 เฟส



รูปที่ 4.8 ขั้นตอนการทำงานของ DES

- เฟสที่ 1 จะทำการจัดการกับข้อมูลที่ไม่ได้ผ่านการเข้ารหัสขนาด 64 บิตผ่านเข้าไปยังส่วนที่เรียกว่า Initial Permutation (IP) ซึ่งจะทำการเรียงเรียงบิตใหม่เพื่อผลิตข้อมูลที่มีการสลับตำแหน่ง
- เฟสที่ 2 จะทำฟังก์ชันเดียวกัน 16 ครั้ง ซึ่งฟังก์ชันนี้รวมการทำ permutation และ substitution ซึ่งผลลัพธ์ที่ได้จากการทำทั้งหมด 16 ครั้งนี้จะได้ข้อมูลขนาด 64 บิตโดยใช้ทั้งข้อมูลที่ไม่ได้ผ่านการเข้ารหัสและคีย์ในการทำ ซึ่งข้อมูลที่มีขนาด 64 บิตที่ได้นี้จะแบ่งเป็น 2 ด้านคือซ้ายและขวา ทั้งหมดจะถูกสลับเปลี่ยนเพื่อผลิต 64 บิตที่เป็น preoutput
- เฟสที่ 3 จะนำ preoutput ผ่านเข้าไปยังส่วนที่เรียกว่า Inverse initial permutation หรือ IP^{-1} ซึ่งทำหน้าที่อินเวอร์สตัวฟังก์ชัน initial permutation ซึ่งทั้งหมดจะผลิต 64 บิตที่เรียกว่าข้อมูลผ่านการเข้ารหัสแล้ว (Ciphertext)

4.3.3 การทำ initial permutation

การทำ initial permutation และการทำ inverse initial permutation จะถูกอธิบายโดยใช้ตารางด้านล่างตามลำดับ ซึ่งจะเห็นได้ว่าฟังก์ชัน permutation ทั้งสองต่างเป็นส่วนกลับซึ่งกันและกัน พิจารณาจาก 64 บิต: M ที่เข้ามา

M_1	M_2	M_3	M_4	M_5	M_6	M_7	M_8	M_9	M_{10}	M_{11}	M_{12}	M_{13}	M_{14}	M_{15}	M_{16}
M_{17}	M_{18}	M_{19}	M_{20}	M_{21}	M_{22}	M_{23}	M_{24}	M_{25}	M_{26}	M_{27}	M_{28}	M_{29}	M_{30}	M_{31}	M_{32}
M_{33}	M_{34}	M_{35}	M_{36}	M_{37}	M_{38}	M_{39}	M_{40}	M_{41}	M_{42}	M_{43}	M_{44}	M_{45}	M_{46}	M_{47}	M_{48}
M_{49}	M_{50}	M_{51}	M_{52}	M_{53}	M_{54}	M_{55}	M_{56}	M_{57}	M_{58}	M_{59}	M_{60}	M_{61}	M_{62}	M_{63}	M_{64}

M_i เป็นตัวเลขฐานสอง เมื่อทำการ permutation $X = IP(M)$ จะได้ดังนี้

M_{58}	M_{50}	M_{42}	M_{34}	M_{26}	M_{18}	M_{10}	M_2	M_{60}	M_{52}	M_{44}	M_{36}	M_{28}	M_{20}	M_{12}	M_4
M_{62}	M_{54}	M_{46}	M_{38}	M_{30}	M_{22}	M_{14}	M_6	M_{64}	M_{56}	M_{48}	M_{40}	M_{32}	M_{24}	M_{16}	M_8
M_{57}	M_{49}	M_{41}	M_{33}	M_{25}	M_{17}	M_9	M_1	M_{59}	M_{51}	M_{43}	M_{35}	M_{27}	M_{19}	M_{11}	M_3
M_{61}	M_{53}	M_{45}	M_{37}	M_{29}	M_{21}	M_{13}	M_5	M_{63}	M_{55}	M_{47}	M_{39}	M_{31}	M_{23}	M_{15}	M_7

ถ้าเราทำการ inverse permutation $Y = IP^{-1}(X) = IP^{-1}(IP(M))$ เราจะสามารถเห็นลำดับในการเรียงของบิตที่มีรูปแบบดังเดิม

4.3.4 รายละเอียดของการทำฟังก์ชันในแต่ละรอบ

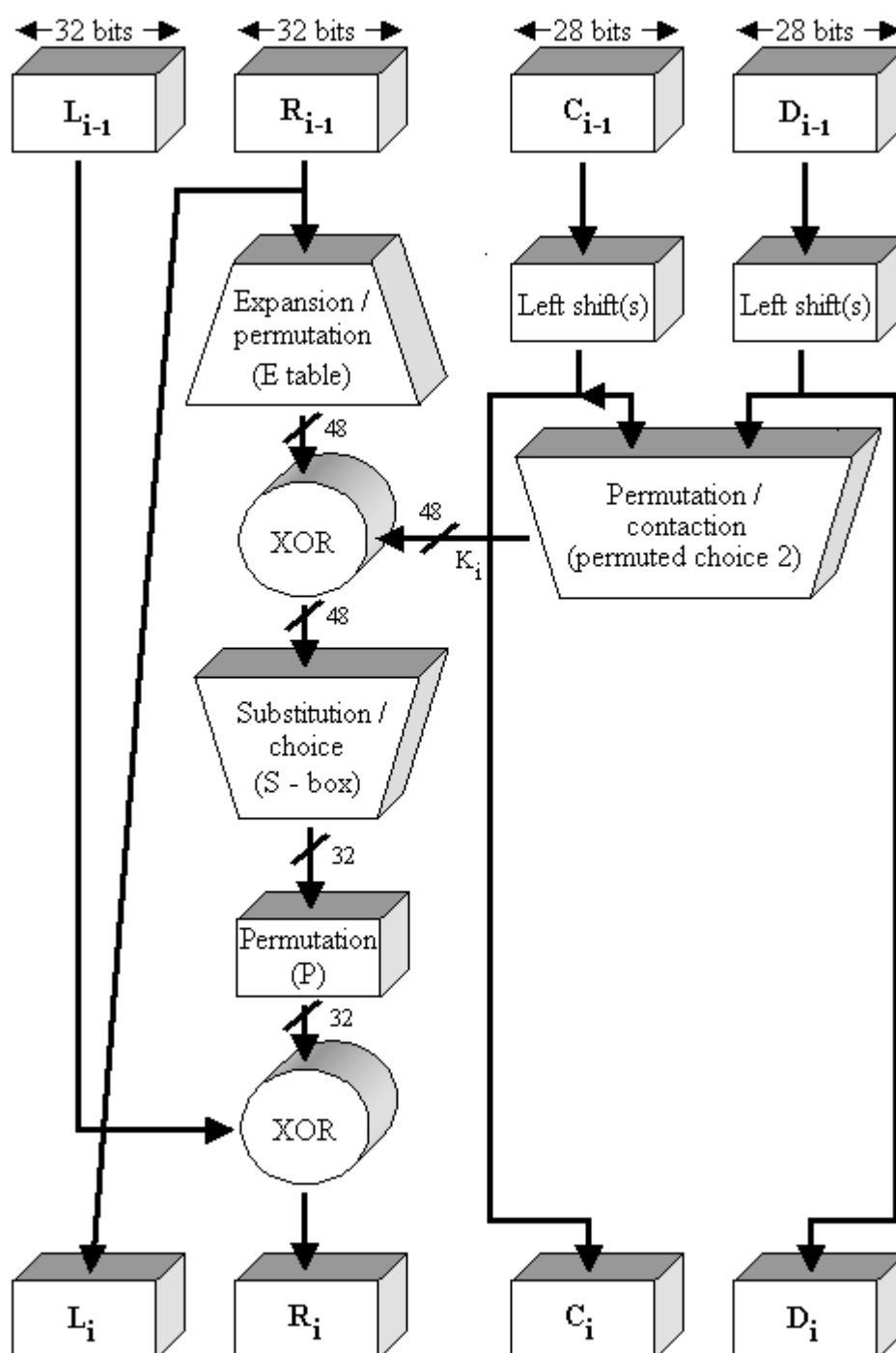
จากข้อมูลที่เข้ามาที่มีขนาด 64 บิตจะทำการแบ่งข้อมูลเป็น 2 ส่วนด้วยกัน ส่วนละขนาด 32 บิต (แบ่งเป็นซ้ายกับขวา) ซึ่งกระบวนการทำในแต่ละครั้งสามารถสรุปเป็นสูตรได้ดังนี้

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

โดย $\oplus$ หมายถึงการทำ XOR function

จากสูตรในข้างต้นจะเห็นได้ว่า 32 บิตด้านซ้ายมือ (L_i) จะเท่ากับด้านขวาของ (R_{i-1}) รอบที่ผ่านมา โดย R_i จะเท่ากับการนำ L_{i-1} มา XOR กับ $f(R_{i-1}, K_i)$ ซึ่งฟังก์ชัน f จะแสดงดังรูปที่ 4.9



รูปที่ 4.9 แสดงการเข้ารหัส DES ในแต่ละครั้ง (ทำทั้งหมด 16 ครั้ง)

(a) Initial Permutation (IP)

Output bit	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Form input bit	58	50	42	34	26	18	10	2	60	52	44	36	28	20	12	4

Output bit	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
Form input bit	62	54	46	38	30	22	14	6	64	56	48	40	32	24	16	8
Output bit	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
Form input bit	57	49	41	33	25	17	9	1	59	51	43	35	27	19	11	3
Output bit	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64
Form input bit	61	53	45	37	29	21	13	5	63	55	47	39	31	23	15	7

(b) Inverse Initial Permutation (IP^{-1})

Output bit	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Form input bit	40	8	48	16	24	24	64	32	39	7	47	15	55	23	63	31
Output bit	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
Form input bit	38	6	46	14	54	22	62	30	37	5	45	13	53	21	61	29
Output bit	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
Form input bit	36	4	44	12	52	20	60	28	35	3	43	11	51	19	59	27
Output bit	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64
Form input bit	34	2	42	10	50	18	58	26	33	1	41	9	49	17	57	25

(c) Expansion Permutation(E)

Output bit	1	2	3	4	5	6	7	8	9	10	11	12
Form input bit	32	1	2	3	4	5	4	5	6	7	8	9
Output bit	13	14	15	16	17	18	19	20	21	22	23	24
Form input bit	8	9	10	11	12	13	12	13	14	15	16	17
Output bit	25	26	27	28	29	30	31	32	33	34	35	36
Form input bit	16	17	18	19	20	21	20	21	22	23	24	25
Output bit	37	38	39	40	41	42	43	44	45	46	47	48
Form input bit	24	25	26	27	28	29	28	29	30	31	32	1

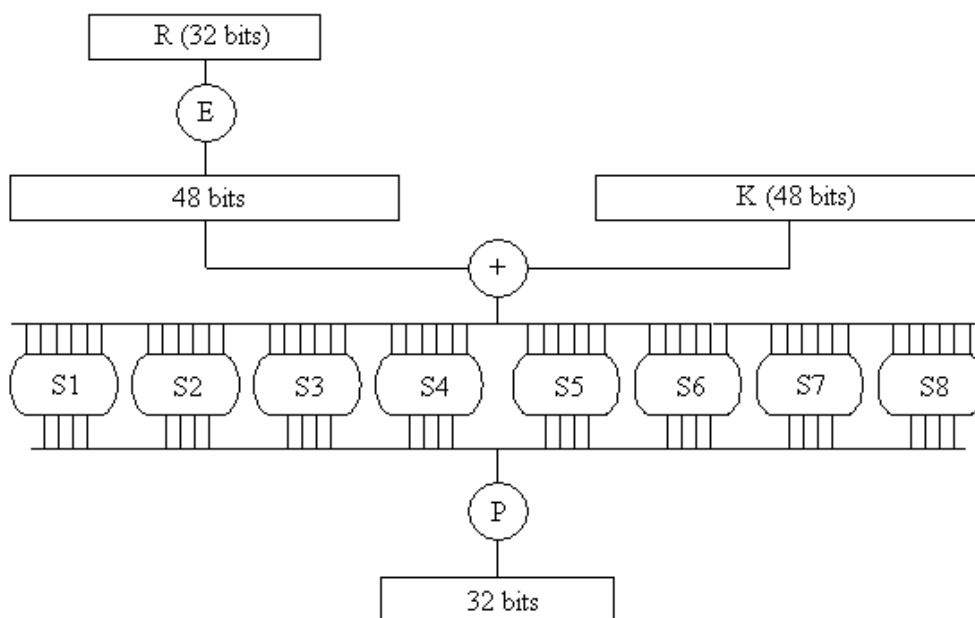
(d) Permutation Function (P)

Output bit	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Form input bit	16	7	20	21	29	12	28	17	1	15	23	26	5	18	31	10
Output bit	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
Form input bit	2	8	24	14	32	27	3	9	19	13	30	6	22	11	4	25

ตารางที่ 4.2 แสดง permutation ของ DES

โดยคีย์ K_1 ที่ใช้ในแต่ละรอบจะมีขนาด 48 บิตและอินพุตด้านขวา (R) มีขนาด 32 บิต ดังนั้น จึงต้องมีการขยายขนาดจาก 32 บิตให้เป็น 48 บิต โดยใช้ตารางที่ได้มีการกำหนด permutation และ

expansion ซึ่งรวมทั้งการจำลอง 16 บิตที่เพิ่มขึ้นมาของด้านขวา ซึ่งจะนำผลที่ได้ที่มีขนาด 48 บิตจะถูก XOR กับ K_1 โดยจะนำผลที่ได้ผ่านไปยังฟังก์ชันที่เรียกว่า Substitution และ Permutation ที่สามารถผลิตผลลัพธ์ที่มีขนาด 32 บิต



รูปที่ 4.10 แสดงการคำนวณ $f(R, K)$

โดย Substitution จะประกอบด้วยเซตของ S – box 8 อัน ($S_1 - S_8$) ซึ่ง S – box จะมีอินพุตขนาด 6 บิตและผลิตเอาต์พุตขนาด 4 บิต ซึ่งจากตารางที่ 4.3 จะแสดง DES S – box โดยมีวิธีการในการแปลงอินพุตขนาด 6 บิตให้กลายเป็นเอาต์พุตขนาด 4 บิตดังนี้คือ การนำบิตแรกและบิตสุดท้ายของอินพุตมาทำเป็นตำแหน่งของแถวและนำ 4 บิตตรงกลางมาเป็นตำแหน่งของคอลัมน์เช่น S_1 มีค่าเท่ากับ 011011 เราจะนำบิตแรกและบิตสุดท้ายซึ่งก็คือ 0 และ 1 มาเป็นตำแหน่งของแถวจะได้แถวที่ 01 และ 4 บิตตรงกลางที่เหลือคือ 1101 จะได้ตำแหน่งของคอลัมน์คือ คอลัมน์ที่ 13 ดังนั้นค่าที่ตำแหน่งแถวที่ 1 และคอลัมน์ที่ 13 ในตารางคือ 0101

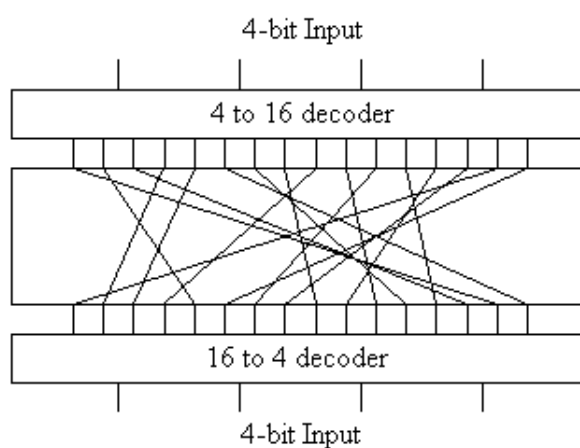
		Column Number															
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Row	Box																
0	S1	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1		0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	0
2		4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3		15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

		Column Number															
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Row	Box																
0	S8	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1		1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
2		7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
3		2	1	14	7	4	10	3	13	15	12	9	0	3	5	6	11

ตารางที่ 4.3 แสดงขั้นตอนใน S-box ทั้ง 8 ชุด

4.3.5 การสร้างคีย์

ในรูปที่ 4.9 ซึ่งแสดงถึงการทำงานในแต่ละรอบของ DES จะเห็นได้ว่าคีย์ที่ใช้มีขนาด 56 บิตเป็นอินพุตในการทำ permutation โดยในตาราง Permuted Choice One เริ่มแรกจะทำการแบ่ง 56 บิตเป็น 2 ส่วนเท่าๆ กันส่วนละ 28 บิตโดยให้ชื่อในแต่ละส่วนว่า C กับ D ซึ่งในแต่ละรอบ (ทั้งหมด 16 รอบ) จะมีการทำ circular left shift ในแต่ละส่วนของ C และ D หรือทำ rotation โดยในแต่ละรอบจะมีการกำหนดว่าจะให้เลื่อนไปที่บิตดังตาราง ซึ่งค่าที่ถูกเลื่อนจะกลายเป็นอินพุตของการทำให้รอบถัดไปและเป็นอินพุตของการทำ Permuted Choice Two ดังในตาราง หลังจากการทำ Permuted Choice Two แล้วจะได้เอาต์พุตขนาด 48 บิต ซึ่งเป็นอินพุตของ $f(R_{i-1}, K_i)$



รูปที่ 4.11 แสดงการทำ *Permutation Choice*

(a) Permuted Choice One (PC-1)

Output bit	1	2	3	4	5	6	7	8	9	10	11	12	13	14
From input bit	57	49	41	33	25	17	9	1	58	50	42	34	26	18
Output bit	15	16	17	18	19	20	21	22	23	24	25	26	27	28
From input bit	10	2	59	51	43	35	27	19	11	3	60	52	44	36

Output bit	29	30	31	32	33	34	35	36	37	38	39	40	41	42
From input bit	63	55	47	39	31	23	15	7	62	54	46	38	30	22
Output bit	43	44	45	46	47	48	49	50	51	52	53	54	55	56
From input bit	14	6	61	53	45	37	29	21	13	5	28	20	12	4

(b) Permuted Choice Two (PC-2)

Output bit	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
From input bit	14	17	11	24	1	5	3	28	15	6	21	10	23	19	12	4
Output bit	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
From input bit	26	8	16	7	27	20	13	2	41	52	31	37	47	55	30	40
Output bit	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
From input bit	51	45	33	48	44	49	39	56	34	53	46	42	50	36	29	32

(c) Schedule of Left Shifts

Iteration number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bits rotated	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

ตารางที่ 4.4 แสดงการสร้างคีย์

4.3.6 การถอดรหัสข้อมูล DES

กระบวนการถอดรหัสโดยใช้ DES นั้นเหมือนกับขั้นตอนในการเข้ารหัส ซึ่งมีขั้นตอนในการทำงานดังนี้คือ นำข้อมูลที่ผ่านมาจากการเข้ารหัสแล้ว (Ciphertext) มาเป็นอินพุตแต่จะมีการใช้คีย์ (K_1) ที่มีลำดับย้อนกลับกับคีย์ที่ใช้ในการเข้ารหัสเช่น K_{16} , K_{15} ... เป็นคีย์แรกและคีย์ถัดไปในการเข้ารหัสแทนดังรูปที่ 4.12 ด้านซ้ายมือจะเป็นขั้นตอนการเข้ารหัสและด้านขวามือเป็นขั้นตอนในการถอดรหัส

เราจะแสดงถึงผลลัพธ์ของขั้นตอนแรกในการกระบวนการถอดรหัส ซึ่งจะเท่ากับ 32 บิตที่ถูกสับเปลี่ยนจากอินพุตของการทำทั้งหมด 16 รอบของการเข้ารหัส เริ่มจาก

$$L_{16} = R_{15}$$

$$R_{16} = L_{15} \oplus f(R_{15}, K_{16})$$

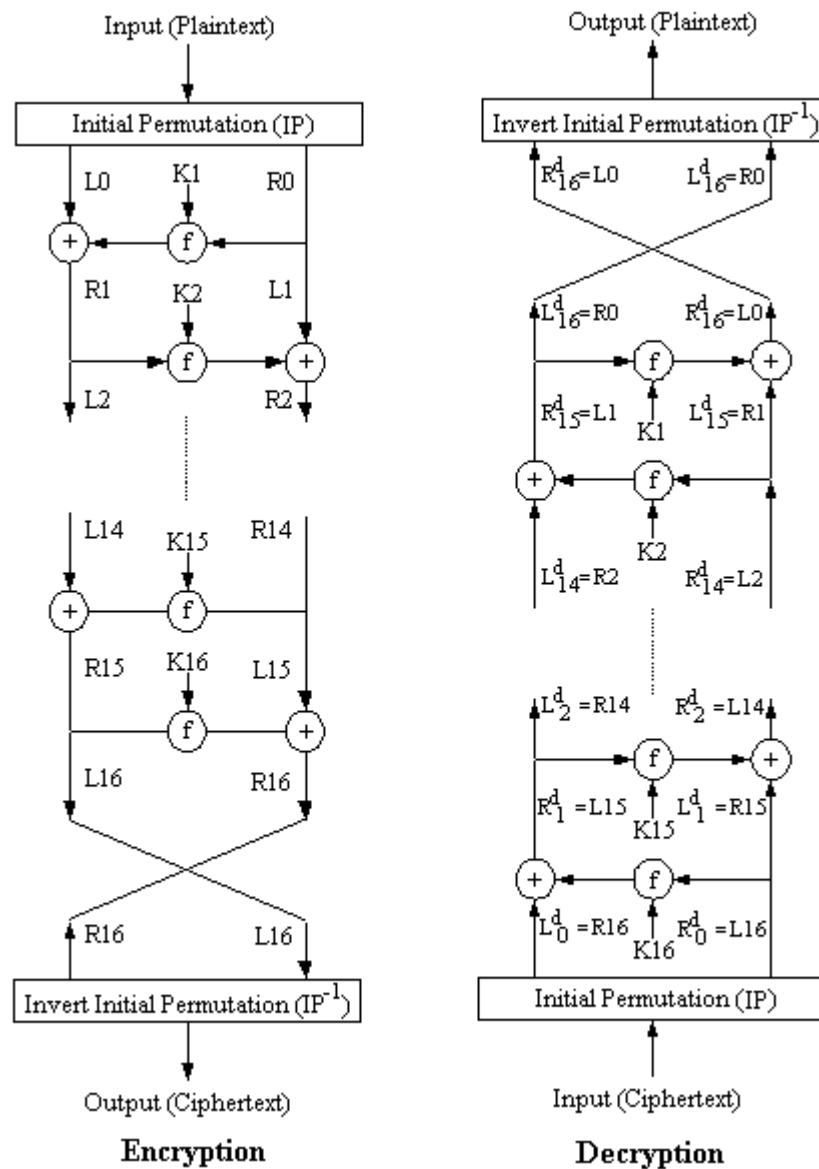
ในด้านการถอดรหัส

$$L_{d1} = R_{d0} = L_{16} = R_{15}$$

$$R_{d1} = L_{d0} \oplus f(R_{d0}, K_{16})$$

$$= R_{16} \oplus f(R_{15}, K_{16})$$

$$= [L_{15} \oplus f(R_{15}, K_{16})] \oplus f(R_{15}, K_{16})$$



รูปที่ 4.12 แสดงคีย์และขั้นตอนการเข้าและถอดรหัส DES

ซึ่งคุณสมบัติของ XOR ที่สำคัญคือ

$$[A \oplus B] \oplus C = A \oplus [B \oplus C]$$

$$D \oplus D = 0$$

$$E \oplus 0 = E$$

ดังนั้นเรามี $L_{d1} = R_{15}$ และ $R_{d1} = L_{15}$ จะได้เอาต์พุตของขั้นตอนแรกในการถอดรหัสคือ $L_{15}||R_{15}$ ซึ่งเราสามารถเขียนเป็นสมการในการถอดรหัสได้ดังนี้คือ

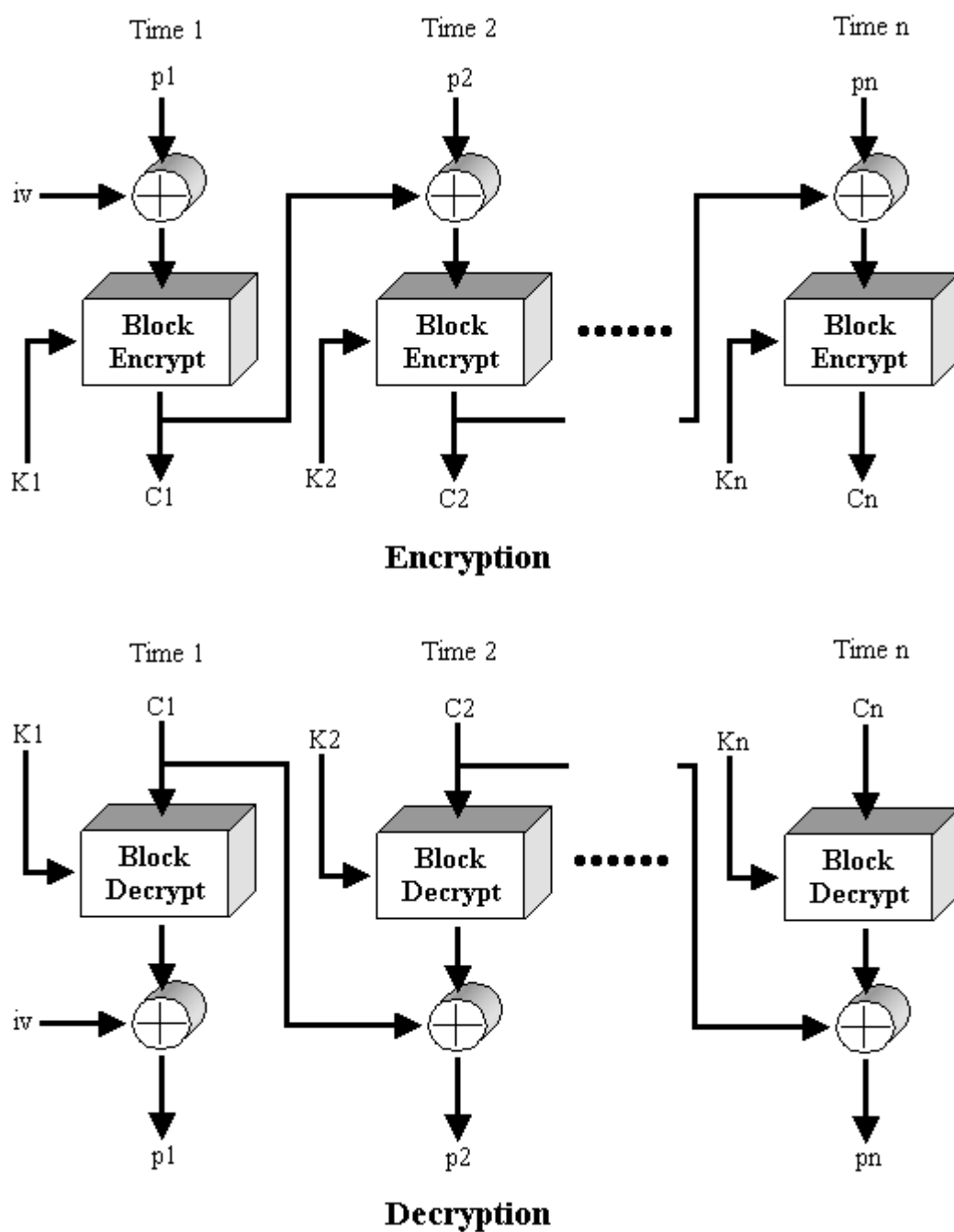
$$R_i^{-1} = L_i$$

$$L_i^{-1} = R_i \oplus f(R_{i-1}, K_i) = R_i \oplus f(L_i, K_i)$$

ซึ่งผลสุดท้ายเอาค่าพหุที่ได้จากขั้นตอนสุดท้ายในการถอดรหัสคือ $R0||L0$ และนำไปสู่ขั้นตอนการทำ Inverse Permutation เราจะได้ข้อมูลที่ส่งมา (Plaintext) ดังสมการ

$$IP^{-1}(L0||R0) = IP^{-1}(IP(Plaintext)) = Plaintext$$

4.3.7 โหมด CBC



รูปที่ 4.13 แสดงการเข้ารหัสและถอดรหัส DES ในโหมด CBC

โหมด CBC (Cipher Block Chaining) เป็นวิธีการเข้ารหัสที่พัฒนามาจาก DES ช่วยทำให้ข้อมูลที่ส่งมีความปลอดภัยมากขึ้น โดยมีวิธีการคือจะนำข้อมูลที่ผ่านมาจากการเข้ารหัสของข้อมูลตัวก่อนมา XOR กับข้อมูลที่ยังไม่ได้เข้ารหัสของตัวถัดไปก่อนจะทำการเข้ารหัสแบบ DES ตามปกติ

ในการถอดรหัสข้อมูลจะทำเช่นเดียวกับการถอดรหัสข้อมูล DES ตามปกติแต่นำผลที่ได้จากการถอดรหัสมา XOR กับข้อมูลที่ผ่านมาจากการเข้ารหัส (Ciphertext) ตัวก่อนหน้าเพื่อจะได้ข้อมูลจริง (Plaintext) ดังสมการ

$$C_n = E_k[C_{n-1} \oplus P_n]$$

สมการในการถอดรหัส

$$D_k[C_n] = D_k[E_k(C_{n-1} \oplus P_n)]$$

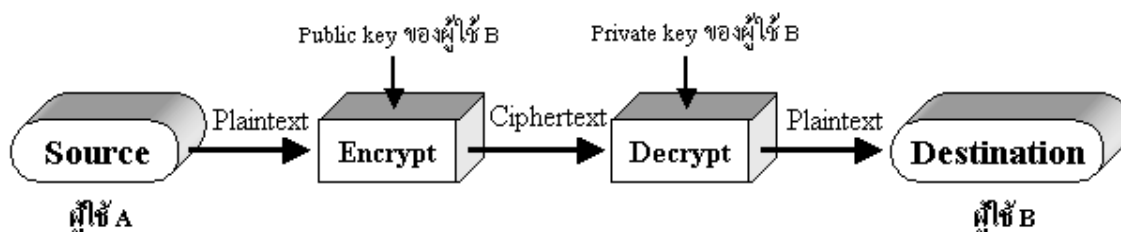
$$D_k[C_n] = C_{n-1} \oplus P_n$$

$$C_{n-1} \oplus D_k[C_n] = C_{n-1} \oplus C_{n-1} \oplus P_n = P_n$$

สังเกตเห็นว่าบล็อกแรกของข้อมูลที่ผ่านการเข้ารหัส (Ciphertext) จะมีการนำ iv (Initialization Vector) มาทำการ XOR กับข้อมูลที่ไม่ได้เข้ารหัส (Plaintext) ก่อนจะมีการเข้ารหัส DES ตามปกติ และในส่วนของการถอดรหัสข้อมูลก็จะใช้ iv ในการถอดรหัสข้อมูลเช่นเดียวกัน ดังนั้นจึงจำเป็นต้องมีข้อตกลงกันระหว่างผู้ส่งกับผู้รับก่อนว่าจะใช้ iv เป็นค่าใด เพื่อให้มีความปลอดภัยสูงสุด iv ควรจะมีการป้องกันเช่นเดียวกับ key ซึ่งในการส่งค่า iv อาจจะทำการส่งโดยใช้การเข้ารหัสแบบ ECB

4.4 การเข้ารหัสแบบ RSA

การเข้ารหัสข้อมูลแบบ RSA ถูกคิดค้นขึ้นในปี ค.ศ.1977 โดยที่ชื่อ RSA ได้มาจากอักษรตัวแรกของนามสกุลของผู้ร่วมกันคิดค้นคือ Ron Rivest, Adi Shamir และ Leonard Adleman ซึ่งเป็นวิธีการเข้ารหัสแบบคีย์สาธารณะที่เรียกว่าพับลิคคีย์ (Public-key Cryptosystem) เพื่อแก้ไขปัญหาคำทำให้คีย์เป็นความลับ (Secret-key Cryptosystem) โดยสมาชิกแต่ละคนจะต้องมีคีย์ 2 ชนิดคือคีย์ส่วนตัวหรือไพรเวตคีย์ (Private key) และคีย์สาธารณะหรือพับลิคคีย์ (Public key) โดยไพรเวตคีย์จะถูกเก็บไว้เป็นความลับ ส่วนพับลิคคีย์นั้นจะเปิดเผยให้บุคคลใดก็ได้ที่ต้องการส่งสารให้แก่ตน ซึ่งมีหลักการการทำงานว่าข้อมูลที่ถูกรหัสด้วยพับลิคคีย์ของบุคคลใดจะสามารถถูกถอดรหัสด้วยไพรเวตคีย์ของบุคคลนั้นได้เท่านั้น



รูปที่ 4.14 แสดงขั้นตอนการทำ Public-key Cryptosystem

4.4.1 หลักการทำงานของ RSA

ถ้าให้ p และ q เป็นจำนวนเฉพาะที่มีค่ามากๆ โดยที่ $n = p \cdot q$ เรียกว่าโมดูลัส (modulus) จากนั้นจึงเลือก e ที่มีค่าน้อยกว่า n และไม่สามารถหาร $(p-1)(q-1)$ ได้ลงตัว ถ้าให้ d เป็นส่วนกลับของ e ในคณิตศาสตร์ระบบโมดูลอฐาน $(p-1)(q-1)$ นั่นคือ

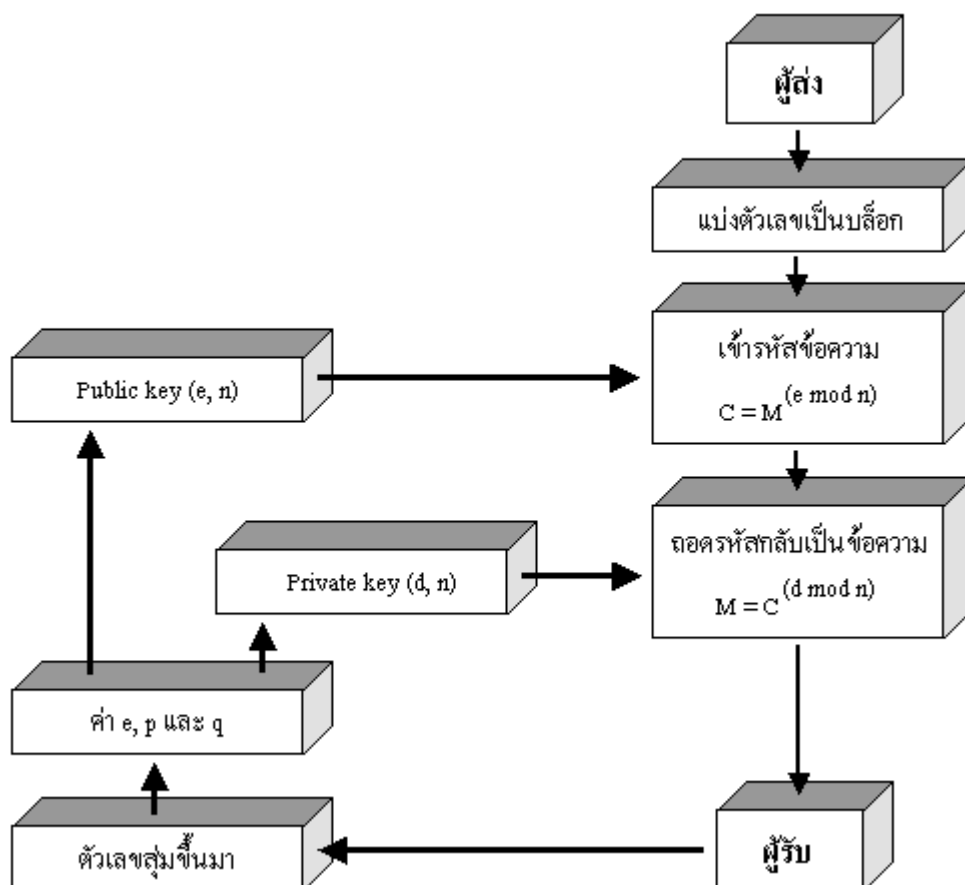
$$e \cdot d \bmod (p-1)(q-1) = 1 \quad \dots\dots\dots(1)$$

ในรหัส RSA นั้น (n, e) คือฟังก์ชันสาธารณะ ส่วน d คือไพรเวตคีย์ เมื่อได้ค่าเหล่านี้แล้ว p, q จะต้องเก็บเป็นความลับหรือถูกทำลายในทันที

ถ้าผู้ใช้ A ต้องการส่งข้อความส่วนตัว m ไปให้ผู้ใช้ B แล้วผู้ใช้ A จะสร้างรหัสลับ c ของ m ได้โดยการให้ $c = m^e \bmod n$ เมื่อ (n, e) เป็นฟังก์ชันสาธารณะของผู้ใช้ B เมื่อผู้ใช้ B ได้รับเอกสารรหัสลับ c ผู้ใช้ B จะถอดรหัสเพื่ออ่านเอกสาร m ได้ด้วย d เพราะความสัมพันธ์ในสมการ (1) ระหว่าง e, d และ n จะทำให้

$$c^d = m^{ed \bmod (p-1)(q-1)} \bmod n = m$$

และเพราะมีแต่ผู้ใช้ B เท่านั้นที่รู้ค่า d ทำให้ผู้ใช้ B เท่านั้นที่จะสามารถถอดรหัสได้



รูปที่ 4.15 แสดงการเข้ารหัสแบบ RSA

ตัวอย่างขั้นตอนการเข้ารหัสแบบ RSA ซึ่งมีขั้นตอนในการหา key ดังนี้

1. เลือกจำนวนเฉพาะสองจำนวน คือ $p = 7, q = 17$

2. คำนวณ $n = p * q = 7 * 17 = 119$
 3. คำนวณ $(p-1)(q-1) = 96$
 4. เลือก e ซึ่งมีความสัมพันธ์กับค่า $(p-1)(q-1)$ ที่ได้กล่าวไว้ข้างต้น ในที่นี้เราจะใช้ 3
 5. คำนวณค่า d ซึ่งสัมพันธ์กับสมการ $e * d = 1 \bmod 96$ ซึ่งค่าที่ถูกต้องคือ $d = 77$ เนื่องจาก

$$77 * 3 = 231 = 2 * 96 + 3$$
- ดังนั้น Public key คือ $\{3, 119\}$ และมีค่า Private key คือ $\{77, 119\}$

4.4.2 การทำลายรหัส RSA

วิธีทำลายรหัส RSA ที่รู้จักกันดีที่สุดคือการหาค่า d นั้นเองจากสมการที่ (1) เราอาจหาค่า d ได้หากรู้ค่า p และ q แต่เนื่องจาก p และ q เป็น prime number ที่ $p * q = n$ ดังนั้นการทำลายรหัส RSA ขึ้นอยู่กับการแยกตัวประกอบของ n นั้นเอง แต่วิธีการแยกตัวประกอบของ n นั้นไม่ง่ายเลยสำหรับ n มีค่ามาก ดังนั้นสำหรับข้อมูลที่มีความสำคัญมากอาจจำเป็นต้องใช้ n ที่มีค่ามากถึง 700 หรือ 1000 บิต

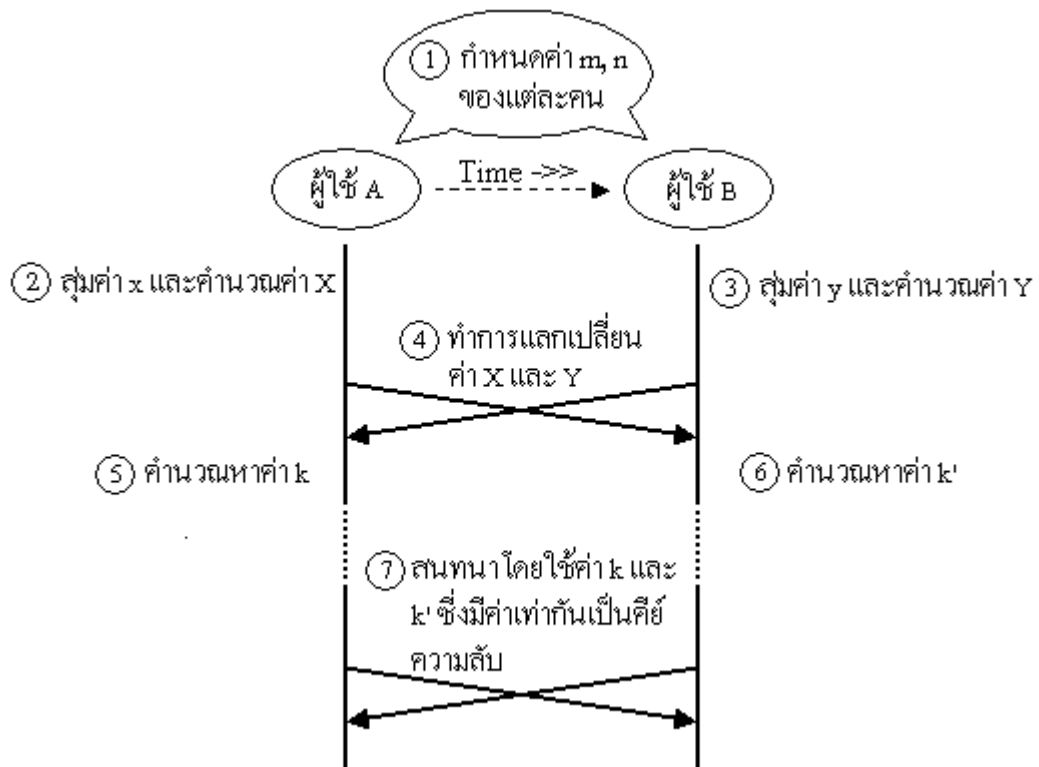
จะเห็นได้ว่าไม่ว่าการเข้ารหัสหรือถอดรหัส RSA ก็จำเป็นต้องใช้การยกกำลังในระบบโมดูลอ ซึ่งการยกกำลังนั้นสามารถทำได้โดยใช้วงจรรวมระบบโมดูลอมาต่อกัน ดังนั้นความเร็วของทั้งการเข้ารหัสและถอดรหัสจึงขึ้นกับความเร็วของวงจรรวมระบบโมดูลอเป็นอย่างมาก ซึ่งเป็นจุดอ่อนข้อหนึ่งของ RSA เมื่อเปรียบเทียบกับคีย์ความลับเช่น DES เพราะปัจจุบันการคูณในระบบโมดูลอยังค่อนข้างยุ่งยากเมื่อเปรียบเทียบกับ การแทนค่าหรือสลับตำแหน่งใน DES ได้มีการประมาณกันว่าสำหรับ n ที่มีความยาว 512 บิต การเข้ารหัสแบบ DES จะเร็วกว่า RSA ประมาณ 10 เท่าถ้าเราทำการเข้ารหัสด้วยซอฟต์แวร์ และอาจเร็วกว่าถึง 1000 ถึง 10000 เท่าหากทำการเข้ารหัสด้วยฮาร์ดแวร์ โดยทั่วไปแล้วเราต้องการให้การเข้ารหัสเร็วกว่าการถอดรหัส ดังนั้นเราจึงมักเลือกให้ e มีค่าน้อยกว่า d และยิ่งกว่านั้นเรายังมักให้ e ของสมาชิกทุกคนมีค่าเดียวกันเพื่อให้ฮาร์ดแวร์ของวงจรเข้ารหัสสำหรับสมาชิกแต่ละคนมีลักษณะคล้ายกัน

เนื่องจาก DES และ RSA มีข้อดีข้อเสียที่แตกต่างกันจึงไม่จำเป็นว่ารหัสชนิดใดชนิดหนึ่งจะเหมาะสมในทุกสถานการณ์ โดยทั่วไปแล้ว DES จะถูกใช้ในการเข้ารหัสข้อมูลที่มีขนาดใหญ่เพราะรวดเร็ว ในขณะที่ RSA จะถูกใช้ในระบบสื่อสารที่ไม่ยาวนานแต่ต้องการความปลอดภัยสูง ในบางครั้ง RSA ยังถูกใช้ร่วมกับ DES เพื่อเสริมจุดเด่นซึ่งกันและกัน เช่นตัวเอกสารจริงจะถูกเข้ารหัสด้วย DES โดยที่คีย์รหัส DES จะถูกเข้ารหัสด้วย RSA แล้วส่งไปด้วยกันหรือส่งไปก่อน แต่ในบางครั้ง DES อย่างเดียวก็เพียงพอแล้วหากการแลกเปลี่ยนคีย์ทำได้อย่างปลอดภัยพอหรือในกรณีเมื่อผู้ส่งและผู้รับเป็นบุคคลเดียวกัน เช่นฮาร์ดดิสก์ในคอมพิวเตอร์ส่วนตัวหรือข้อมูลส่วนตัวในสมาร์ตการ์ด

4.5 การสร้างคีย์ความลับด้วยวิธี Diffie-Hellman

Diffie-Hellman ซึ่งถือกำเนิดขึ้นในปี ค.ศ.1976 โดย Whitfield Diffie และ Martin Hellman เป็นวิธีที่ใช้ในการแลกเปลี่ยนคีย์ระหว่างผู้ใช้งาน 2 คนแล้วนำค่าทั้งสองมาสร้างเป็นคีย์ความลับสำหรับใช้ในการเข้ารหัสข้อมูลในการสื่อสารระหว่างกัน โดยลำดับขั้นตอนในการสร้างคีย์ความลับด้วยวิธี Diffie-Hellman ต่อไปนี้จะแสดงถึงการสร้างคีย์ความลับระหว่างผู้ใช้ A กับผู้ใช้ B ซึ่งมีขั้นตอนดังนี้

1. ผู้ใช้ A และผู้ใช้ B กำหนดค่า m และ n โดยที่ $1 < n < m$ เลขทั้งสองไม่จำเป็นต้องปกปิด
2. ผู้ใช้ A สุ่มตัวเลขที่มีค่ามากๆ มาตัวหนึ่งกำหนดให้เป็นค่า x แล้วหาค่า $X = n^x \bmod m$ และให้ผู้ใช้ A เก็บค่า x เอาไว้เป็นความลับ
3. ให้ผู้ใช้ B ทำเหมือนกับผู้ใช้ A ในข้อ 2 สุ่มตัวเลขที่มีค่ามากๆ มาตัวหนึ่งกำหนดให้เป็นค่า y แล้วหาค่า $Y = n^y \bmod m$ และให้ผู้ใช้ B เก็บค่า y เป็นความลับ
4. ให้ผู้ใช้ A และผู้ใช้ B ทำการแลกค่า X และ Y กัน
5. ผู้ใช้ A คำนวณหาค่า $k = Y^x \bmod m$
6. ผู้ใช้ B คำนวณหาค่า $k' = X^y \bmod m$
7. ผู้ใช้ A และผู้ใช้ B ใช้ค่า k และ k' ซึ่งมีค่าเท่ากันคือ $n^{xy} \bmod m$ เป็นคีย์ความลับในการใช้งานร่วมกัน



รูปที่ 4.16 ขั้นตอนการแลกเปลี่ยนคีย์ด้วยวิธี Diffie-Hellman

ในการสร้างคีย์ด้วยวิธี Diffie-Hellman นี้ก็เหมือนกับการใช้งานพับลิคคีย์และไพรเวตคีย์นั่นเอง ซึ่งเราสังเกตได้ว่าค่า x และ y จะเป็นค่าที่ถูกเก็บไว้กับผู้ใช้เท่านั้นซึ่งก็คือไพรเวตคีย์ ส่วนค่า X และ Y เป็นค่าที่ส่งไปให้กับผู้ใช้ที่ต้องการทำการติดต่อดังซึ่งก็คือพับลิคคีย์นั่นเอง โดยค่าคีย์ความลับที่จะใช้ในการนำมาใช้ในการเข้ารหัสระหว่างผู้ใช้งานทั้งสองฝั่งนั้น จะเป็นการนำค่าไพรเวตคีย์ของตนและพับลิคคีย์ของผู้ใช้ที่ต้องการติดต่อดังมาสร้างเป็นค่า k และ k' ซึ่งมีค่าเท่ากันมาใช้เป็นคีย์ความลับในการติดต่อกัน

ค่า k และ k' ที่คำนวณได้นั้นนอกจากผู้ใช้ A และผู้ใช้ B แล้วบุคคลอื่นจะไม่สามารถหาค่าได้ เพราะค่าที่บุคคลอื่นสามารถทราบได้มีเพียงค่า m , n , X และ Y โอกาสที่จะหาค่า x จากค่า X หรือค่า y จากค่า Y นั้นทำได้ด้วยการหาอินเวอร์สของ X ซึ่งเรียกว่า discrete logarithm ซึ่ง discrete logarithm นี้ไม่ได้คำนวณหาได้ง่าย และในการคำนวณ discrete logarithm ในบางค่า X หรือ Y อาจจะไม่มีความเป็นไปได้